

Chapter 11 — SQL Functions

11.1 What Are SQL Functions?

SQL functions are built-in operations that process values and return a result.

Basic Syntax

```
FUNCTION_NAME(value);
```

Example:

```
SELECT UPPER('hello');
```

Output:

```
HELLO
```

Categories Covered

1. String Functions
2. Numeric Functions
3. Date and Time Functions
4. Aggregate Functions
5. NULL-Handling Functions
6. Conditional Functions
7. Conversion Functions
8. System / Information Functions
9. JSON Functions

Scope boundary: Detailed GROUP BY/HAVING, JOINS, subqueries, set operators, views, indexes, transactions, and window functions are reserved for later chapters.

11.2 String Functions

String functions manipulate text values.

UPPER()

```
SELECT UPPER('hello world');  
SELECT UPPER(name) AS uppercase_name FROM employees;
```

LOWER()

```
SELECT LOWER('HELLO WORLD');
```

LENGTH()

Returns string length in bytes.

```
SELECT LENGTH('Hello');
```

CHAR_LENGTH()

Returns number of characters.

```
SELECT CHAR_LENGTH('Hello');
```

``LENGTH()`` counts bytes; ``CHAR_LENGTH()`` counts characters.

CONCAT()

```
SELECT CONCAT('Shaik', ' ', 'Rameez');  
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM employees;
```

CONCAT_WS()

```
SELECT CONCAT_WS('-', '2026', '09', '17');
```

SUBSTRING()

```
SELECT SUBSTRING('DATABASE', 1, 4);
```

Syntax: ``SUBSTRING(string, start, length)``

LEFT()

```
SELECT LEFT('DATABASE', 4);
```

RIGHT()

```
SELECT RIGHT('DATABASE', 4);
```

TRIM()

```
SELECT TRIM(' Hello ');
```

LTRIM()

```
SELECT LTRIM(' Hello');
```

RTRIM()

```
SELECT RTRIM('Hello ');
```

REPLACE()

```
SELECT REPLACE('I like Java', 'Java', 'SQL');
```

REVERSE()

```
SELECT REVERSE('SQL');
```

LOCATE()

```
SELECT LOCATE('SQL', 'I am learning SQL');
```

POSITION()

```
SELECT POSITION('SQL' IN 'I am learning SQL');
```

INSTR()

```
SELECT INSTR('I am learning SQL', 'SQL');
```

LPAD()

```
SELECT LPAD('123', 5, '0');
```

Output: ``00123``

RPAD()

```
SELECT RPAD('123', 5, '0');
```

Output: ``12300``

11.3 Numeric Functions

ABS()

```
SELECT ABS(-100);
```

CEIL() / CEILING()

```
SELECT CEIL(10.2);  
SELECT CEILING(10.2);
```

FLOOR()

```
SELECT FLOOR(10.9);
```

ROUND()

```
SELECT ROUND(10.567, 2);
```

TRUNCATE()

```
SELECT TRUNCATE(10.567, 2);
```

Difference:

```
ROUND(10.567, 2)    → 10.57  
TRUNCATE(10.567, 2) → 10.56
```

MOD()

```
SELECT MOD(10, 3);  
SELECT 10 % 3;
```

POWER()

```
SELECT POWER(2, 3);
```

SQRT()

```
SELECT SQRT(25);
```

SIGN()

```
SELECT SIGN(-50);
```

Returns `1` for positive, `0` for zero, and `-1` for negative.

RAND()

```
SELECT RAND();
```

GREATEST()

```
SELECT GREATEST(10, 25, 15);
```

LEAST()

```
SELECT LEAST(10, 25, 15);
```

11.4 Date and Time Functions

NOW()

```
SELECT NOW();
```

CURRENT_DATE()

```
SELECT CURRENT_DATE();
```

CURRENT_TIME()

```
SELECT CURRENT_TIME();
```

CURRENT_TIMESTAMP()

```
SELECT CURRENT_TIMESTAMP();
```

DATE()

```
SELECT DATE('2026-09-17 18:30:45');
```

TIME()

```
SELECT TIME('2026-09-17 18:30:45');
```

YEAR()

```
SELECT YEAR('2026-09-17');
```

MONTH()

```
SELECT MONTH('2026-09-17');
```

MONTHNAME()

```
SELECT MONTHNAME('2026-09-17');
```

DAY()

```
SELECT DAY('2026-09-17');
```

DAYNAME()

```
SELECT DAYNAME('2026-09-17');
```

DAYOFWEEK()

```
SELECT DAYOFWEEK('2026-09-17');
```

HOUR(), MINUTE(), SECOND()

```
SELECT HOUR('18:30:45');  
SELECT MINUTE('18:30:45');  
SELECT SECOND('18:30:45');
```

DATE_ADD()

```
SELECT DATE_ADD('2026-09-17', INTERVAL 10 DAY);  
SELECT DATE_ADD(hire_date, INTERVAL 1 YEAR) FROM employees;
```

DATE_SUB()

```
SELECT DATE_SUB('2026-09-17', INTERVAL 10 DAY);
```

DATEDIFF()

Returns date difference in days.

```
SELECT DATEDIFF('2026-09-17', '2026-09-01');
```

TIMESTAMPDIFF()

```
SELECT TIMESTAMPDIFF(YEAR, '2000-05-10', '2026-09-17');
```

DATE_FORMAT()

```
SELECT DATE_FORMAT('2026-09-17', '%d-%m-%Y');
SELECT DATE_FORMAT(hire_date, '%d-%m-%Y') FROM employees;
```

Common specifiers:

- `%Y` — 4-digit year
- `%m` — month number
- `%d` — day
- `%M` — month name
- `%W` — weekday name

11.5 Aggregate Functions

Aggregate functions operate across multiple rows and return an aggregate result.

Detailed GROUP BY and HAVING concepts are taught later.

COUNT()

```
SELECT COUNT(*) FROM employees;
SELECT COUNT(email) FROM employees;
```

`COUNT(\*)` counts rows. `COUNT(column)` ignores NULL values in that column.

SUM()

```
SELECT SUM(salary) FROM employees;
```

AVG()

```
SELECT AVG(salary) FROM employees;
```

MIN()

```
SELECT MIN(salary) FROM employees;
```

MAX()

```
SELECT MAX(salary) FROM employees;
```

Combined:

```
SELECT COUNT(*) AS total_employees,
       SUM(salary) AS total_salary,
       AVG(salary) AS average_salary,
       MIN(salary) AS minimum_salary,
       MAX(salary) AS maximum_salary
FROM employees;
```

11.6 NULL-Handling Functions

IFNULL()

```
SELECT IFNULL(NULL, 'Not Available');
```

```
SELECT name, IFNULL(phone, 'No Phone') AS phone
FROM customers;
```

COALESCE()

Returns the first non-NULL value.

```
SELECT COALESCE(NULL, NULL, 'Hello', 'World');
```

```
SELECT name,  
       COALESCE(phone, email, 'No Contact') AS contact  
FROM customers;
```

NULLIF()

Returns NULL when two expressions are equal; otherwise returns the first expression.

```
SELECT NULLIF(10, 10);  
SELECT NULLIF(10, 20);
```

11.7 Conditional Functions

IF()

Syntax:

```
`IF(condition, value_if_true, value_if_false)`
```

```
SELECT IF(80 >= 40, 'Pass', 'Fail');
```

```
SELECT name, marks,  
       IF(marks >= 40, 'Pass', 'Fail') AS result  
FROM students;
```

CASE

Allows multiple conditions.

```
SELECT name, marks,  
       CASE  
         WHEN marks >= 90 THEN 'A'  
         WHEN marks >= 75 THEN 'B'  
         WHEN marks >= 60 THEN 'C'  
         WHEN marks >= 40 THEN 'D'  
         ELSE 'F'  
       END AS grade  
FROM students;
```

11.8 Conversion Functions

CAST()

```
SELECT CAST('100' AS UNSIGNED);  
SELECT CAST(price AS DECIMAL(10,2)) FROM products;
```

Common types include `SIGNED`, `UNSIGNED`, `DECIMAL`, `CHAR`, `DATE`, and `DATETIME`.

CONVERT()

```
SELECT CONVERT('100', UNSIGNED);  
SELECT CONVERT('2026-09-17', DATE);
```

11.9 System / Information Functions

VERSION()

```
SELECT VERSION();
```

DATABASE()

```
SELECT DATABASE();
```

USER()

```
SELECT USER();
```

CONNECTION_ID()

```
SELECT CONNECTION_ID();
```

11.10 JSON Functions

This section is an introduction to MySQL JSON functions.

JSON_OBJECT()

```
SELECT JSON_OBJECT('name', 'Ravi', 'age', 22);
```

JSON_ARRAY()

```
SELECT JSON_ARRAY('Java', 'SQL', 'Python');
```

JSON_EXTRACT()

```
SELECT JSON_EXTRACT(
    '{"name": "Ravi", "age": 22}',
    '$.name'
);
```

11.11 Scalar Functions vs Aggregate Functions

Scalar Functions

Operate on individual values and normally produce a result for each row.

```
SELECT UPPER(name)
FROM employees;
```

Aggregate Functions

Operate across multiple rows and produce an aggregate result.

```
SELECT AVG(salary)
FROM employees;
```

| Type | Examples | Works With |

|---|---|---|

| String | UPPER(), LOWER() | Text |

| Numeric | ROUND(), ABS() | Numbers |

| Date/Time | YEAR(), DATE_ADD() | Dates |

| NULL | IFNULL(), COALESCE() | NULL values |

| Conditional | IF(), CASE | Conditions |

| Conversion | CAST(), CONVERT() | Data types |

| Aggregate | COUNT(), SUM(), AVG() | Multiple rows |

| JSON | JSON_OBJECT(), JSON_EXTRACT() | JSON |

11.12 Functions with SELECT

```
SELECT name,  
       UPPER(name) AS uppercase_name,  
       LENGTH(name) AS name_length  
FROM employees;
```

11.13 Functions with WHERE

```
SELECT *  
FROM employees  
WHERE UPPER(first_name) = 'RAVI';
```

```
SELECT *  
FROM employees  
WHERE YEAR(hire_date) = 2025;
```

11.14 Functions with ORDER BY

```
SELECT first_name, salary  
FROM employees  
ORDER BY ROUND(salary, 0) DESC;
```

11.15 Basic Functions with GROUP BY

```
SELECT department,  
       COUNT(*) AS employee_count  
FROM employees  
GROUP BY department;  
  
SELECT department,  
       AVG(salary) AS average_salary  
FROM employees  
GROUP BY department;
```

Detailed GROUP BY and HAVING will be covered in a later chapter.

11.16 Combining Multiple Functions

```
SELECT CONCAT(UPPER(first_name), ' ', UPPER(last_name)) AS full_name,  
       ROUND(salary, 2) AS salary,  
       DATE_FORMAT(hire_date, '%d-%m-%Y') AS joining_date  
FROM employees;
```

11.17 Practical Problems

1. Display employee names in uppercase.

```
SELECT UPPER(name) FROM employees;
```

2. Display names and their lengths.

```
SELECT name, LENGTH(name) AS name_length  
FROM employees;
```

3. Round salaries to two decimal places.

```
SELECT name, ROUND(salary, 2) AS rounded_salary  
FROM employees;
```

4. Find the highest salary.

```
SELECT MAX(salary) AS highest_salary  
FROM employees;
```

5. Find average salary.

```
SELECT AVG(salary) AS average_salary
FROM employees;
```

6. Count employees.

```
SELECT COUNT(*) AS total_employees
FROM employees;
```

7. Display employees who joined in 2025.

```
SELECT *
FROM employees
WHERE YEAR(hire_date) = 2025;
```

8. Display joining dates as DD-MM-YYYY.

```
SELECT name,
       DATE_FORMAT(hire_date, '%d-%m-%Y') AS joining_date
FROM employees;
```

9. Replace NULL phone numbers.

```
SELECT name,
       IFNULL(phone, 'Not Available') AS phone
FROM employees;
```

10. Display salary category.

```
SELECT name, salary,
       CASE
         WHEN salary >= 100000 THEN 'High'
         WHEN salary >= 50000 THEN 'Medium'
         ELSE 'Low'
       END AS salary_category
FROM employees;
```

11.18 Interview Questions

1. What is an SQL function?
2. What are scalar functions?
3. What are aggregate functions?
4. Difference between COUNT(*) and COUNT(column)?
5. Difference between LENGTH() and CHAR_LENGTH()?
6. Difference between ROUND() and TRUNCATE()?
7. Difference between CEIL() and FLOOR()?
8. Difference between NOW() and CURRENT_DATE()?
9. What does DATEDIFF() return?
10. Difference between IFNULL() and COALESCE()?
11. What is NULLIF()?
12. What is CASE in SQL?

13. Difference between CAST() and CONVERT()?
14. Can functions be used in WHERE?
15. Can functions be used in ORDER BY?
16. Can aggregate functions be used with GROUP BY?
17. Difference between scalar and aggregate functions?
18. How do you find the maximum salary?
19. How do you calculate average salary?
20. How do you extract the year from a date?
21. How do you replace NULL values?
22. How do you concatenate first and last names?
23. How do you format a date?
24. How do you calculate the number of days between two dates?

11.19 Chapter Summary

This chapter covered SQL functions used to process, transform, calculate, format, and analyze values.

Covered

- String Functions
- Numeric Functions
- Date and Time Functions
- Aggregate Functions
- NULL-Handling Functions
- Conditional Functions
- Conversion Functions
- System / Information Functions
- JSON Functions
- Scalar vs Aggregate Functions
- Functions with SELECT
- Functions with WHERE
- Functions with ORDER BY
- Basic functions with GROUP BY
- Combining Multiple Functions
- Practical Problems
- Interview Questions

Topics Reserved for Later Chapters

GROUP BY & HAVING → separate chapter
JOINS → separate chapter
Subqueries → separate chapter

Set Operators	→ separate chapter
Views	→ separate chapter
Indexes	→ separate chapter
Transactions	→ separate chapter
Window Functions	→ separate chapter